DR. STONE: PROJETO DE UMA LINGUAGEM DE DOMÍNIO ESPECÍFICO PARA IDENTIFICAÇÃO DE MINERAIS

Ana Luiza S. · Kauan Santos · Riglei Marcos · Rubens Bolognesi · Vagner Pires

Projeto e Implementação de Compiladores — Prof. Marnes Cassule

2026

RESUMO

Este artigo descreve a especificação e a arquitetura do Dr. Stone, uma linguagem de domínio específico (DSL) destinada à representação de amostras minerais e à solicitação de sua identificação por comparação de propriedades físicas. O trabalho, desenvolvido no contexto da disciplina de Projeto e Implementação de Compiladores, investiga a aplicação didática de análise léxica, análise sintática, árvore sintática abstrata, análise semântica e interpretação. A metodologia consiste na análise documental da especificação da linguagem, da apresentação do projeto e da organização de seu repositório público. A linguagem utiliza comandos declarativos, arquivos com extensão .stone e propriedades como cor, dureza, densidade e brilho. O projeto prevê validações de tipos e limites físicos e um catálogo de referência para comparação de características. O repositório descreve uma arquitetura modular em Python com lexer e parser recursivo descendente manuais. Os exemplos documentados demonstram a expressividade da sintaxe e ilustram condições de aceitação e rejeição semântica; não constituem, por si sós, comprovação de desempenho ou precisão mineralógica. Conclui-se que o Dr. Stone oferece um estudo de caso acessível para relacionar fundamentos de compiladores a um domínio científico, preservando limites claros entre compatibilidade de atributos e diagnóstico laboratorial.

Palavras-chave: Compiladores. Linguagem de domínio específico. Análise semântica. Mineralogia. Python.

ABSTRACT

This paper describes the specification and architecture of Dr. Stone, a domain-specific language (DSL) designed to represent mineral samples and request their identification through the comparison of physical properties. Developed for a Compiler Design and Implementation course, the project explores the educational application of lexical analysis, parsing, abstract syntax trees, semantic analysis, and interpretation. The methodology is based on a documentary

examination of the language specification, project presentation, and public repository structure. Dr. Stone uses declarative commands, .stone files, and attributes such as color, hardness, density, and luster. The proposed processing flow validates data and compares sample attributes with a reference catalog. The repository describes a modular Python architecture with a manually implemented lexer and recursive-descent parser. Documented examples illustrate valid and invalid inputs but do not establish measured identification accuracy or runtime performance. The project provides a practical case study connecting compiler fundamentals to a scientific domain while distinguishing attribute compatibility from laboratory diagnosis.
Keywords: Compilers. Domain-specific language. Semantic analysis. Mineralogy. Python.

1 INTRODUÇÃO

Linguagens de programação permitem expressar problemas por meio de regras formais que podem ser reconhecidas e processadas computacionalmente. Além das linguagens de propósito geral, linguagens de domínio específico (DSLs) restringem seu vocabulário e suas operações a uma área de aplicação, favorecendo representações próximas dos conceitos utilizados pelos especialistas do domínio (FOWLER, 2010).

No estudo de compiladores, a separação entre análise léxica, análise sintática e análise semântica permite observar como um texto de entrada é transformado em estruturas interpretáveis. A construção de uma DSL reduzida é, portanto, uma estratégia didática para integrar conceitos formais e implementação prática (AHO et al., 2006).

O Dr. Stone foi concebido para descrever amostras minerais mediante propriedades como cor, dureza, densidade e brilho e solicitar uma identificação por compatibilidade com um catálogo. A questão central deste artigo é: como estruturar uma DSL declarativa, com regras sintáticas e semânticas explícitas, para tornar observável o pipeline de um compilador/interpretador em um cenário de mineralogia?

O objetivo geral é apresentar e analisar a especificação e a arquitetura do Dr. Stone. Como objetivos específicos, busca-se caracterizar a gramática, descrever os módulos do processamento, explicitar as regras de validação, examinar exemplos documentados e identificar limites e oportunidades de evolução.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Linguagens de domínio específico

Uma DSL é projetada para um conjunto delimitado de tarefas e conceitos, diferentemente de linguagens de uso geral. No Dr. Stone, termos como “mineral”, “cor”, “dureza” e “identificar” expressam diretamente operações do domínio, reduzindo a necessidade de construções genéricas. A simplicidade sintática é deliberada: a versão inicial não contempla expressões aritméticas, laços, condicionais ou funções definidas pelo usuário.

2.2 Etapas de um compilador/interpretador

A análise léxica reconhece unidades significativas do código-fonte, denominadas tokens. A análise sintática verifica a organização desses tokens segundo uma gramática e pode construir uma árvore sintática abstrata (AST). A análise semântica avalia restrições que não são suficientemente expressas pela gramática, como referências válidas, tipos e faixas permitidas. Por fim, um interpretador percorre a representação interna para executar comandos e produzir resultados (AHO et al., 2006). A construção de analisadores sintáticos descendentes e suas estratégias de reconhecimento são discutidas por Grune e Jacobs (2008).

2.3 Propriedades físicas na descrição de minerais

A identificação mineralógica pode considerar propriedades observáveis e mensuráveis. A dureza frequentemente é expressa pela escala de Mohs, enquanto densidade, cor e brilho oferecem pistas adicionais. Contudo, a coincidência de características não determina necessariamente uma identificação inequívoca; propriedades podem se sobrepor entre materiais e exigir exames complementares. O Dr. Stone utiliza esse contexto como domínio de aplicação didático, não como substituto de procedimentos gemológicos.

3 METODOLOGIA

Trata-se de um estudo descritivo e aplicado, apoiado em análise documental. Foram considerados a apresentação acadêmica “Dr. Stone: uma linguagem científica para identificar minerais, gemas e joias”, a especificação pública ESPECIFICACAO.md (versão inicial 0.1) e o README do repositório DrStone, consultados em 27 de outubro de 2026. A análise privilegia os

comportamentos explicitamente descritos nas fontes e separa funcionalidades planejadas de resultados efetivamente demonstrados.

As etapas analíticas foram: (a) levantamento dos comandos e atributos; (b) descrição das regras lexicais e gramaticais; (c) mapeamento da arquitetura modular; (d) comparação de exemplos de entrada válida e inválida; e (e) identificação de limitações e critérios para uma futura avaliação empírica. Não foram realizadas medições independentes de tempo de execução, acurácia ou cobertura de testes para este artigo.

4 ESPECIFICAÇÃO DA LINGUAGEM DR. STONE

4.1 Sintaxe declarativa

Um programa contém declarações de amostras e comandos de identificação. A especificação define arquivos UTF-8 com extensão .stone, diferencia maiúsculas e minúsculas e utiliza quebras de linha para separar propriedades e comandos. O sinal de igualdade atribui um valor a uma propriedade, não representa comparação lógica. Comentários começam com o caractere #.

Quadro 1 — Exemplo de programa Dr. Stone

```
mineral amostra1 {
   cor = "incolor"
   dureza = 7
   densidade = 2.65
   brilho = "vitreo"
}
identificar amostra1
```

Fonte: adaptado da especificação do projeto DrStone (2026).

O comando mineral introduz uma amostra nomeada e um bloco de propriedades. O comando identificar solicita sua comparação com o catálogo. A versão 0.1 aceita literais numéricos inteiros ou decimais com ponto e valores textuais entre aspas duplas. A forma léxica de identificadores segue, de maneira simplificada, [A-Za-z_][A-Za-z0-9_]*.

4.2 Gramática e restrições estruturais

A documentação utiliza notação semelhante à EBNF. Em síntese, o programa é uma sequência de comandos; cada comando corresponde a uma declaração mineral ou solicitação de identificação; e uma propriedade é composta por identificador, sinal de atribuição e valor numérico ou textual.

Quadro 2 — Fragmento simplificado da gramática

```
programa          = { fim_de_linha | comando }, EOF ;
comando           = declaracao_mineral | comando_identificar ;
propriedade       = identificador, "=", valor ;
comando_identificar = "identificar", identificador, [ fim_de_linha ] ;
valor             = numero | string ;
```

Fonte: adaptado de DrStone, ESPECIFICACAO.md (2026).

A gramática define a forma dos comandos, mas certas condições são verificadas semanticamente: a amostra deve ser declarada antes de sua identificação, nomes de amostras devem ser únicos, propriedades não podem ser repetidas e cada amostra deve apresentar ao menos uma propriedade.

4.3 Validação semântica

A apresentação do projeto estabelece as seguintes regras para os quatro atributos principais: cor e brilho recebem texto; dureza recebe número entre 1 e 10; densidade recebe número maior ou igual a zero. Essas condições exemplificam a diferença entre validade sintática e validade semântica: a instrução dureza = 11 possui forma válida, mas infringe o intervalo permitido.

Atributo	Tipo	Regra documentada
cor	Texto	Valor textual
brilho	Texto	Valor textual
dureza	Número	Entre 1 e 10
densidade	Número	Maior ou igual a zero

Fonte: apresentação Dr. Stone, página 7 (2026).

5 ARQUITETURA DO COMPILADOR/INTERPRETADOR

O README do repositório descreve uma implementação modular em Python, organizada no fluxo: arquivo .stone → lexer → tokens → parser → AST → análise semântica → interpretador → resultado. O parser é descrito como recursivo descendente, implementado manualmente. Essa arquitetura é a referência técnica adotada neste artigo. A documentação oficial do Python (PYTHON SOFTWARE FOUNDATION, 2026) constitui referência complementar para o ambiente de implementação.

Módulo	Responsabilidade descrita
tokens.py	Definição de tipos e dados dos tokens
lexer.py	Conversão do texto-fonte em tokens
parser.py	Verificação da gramática e construção da AST
ast_nodes.py	Representação dos nós da AST
semantic.py	Verificação de regras e referências
interpreter.py	Execução dos comandos da AST

minerals.py	Catálogo de referências minerais
main.py	Coordenação das etapas e entrada do programa

Fonte: repositório DrStone, README.md (2026).

O catálogo é descrito no README como composto por 100 gemas ou variedades e seis referências genéricas adicionais. Essa informação caracteriza a organização declarada do projeto; não equivale a uma validação independente da abrangência ou da qualidade mineralógica da base.

A apresentação final menciona Python e Lark como plano tecnológico. Entretanto, o README público descreve lexer e parser manuais. Por se tratar de fontes com ênfases diferentes, o artigo registra essa divergência como evolução ou diferença entre proposta e implementação documentada, sem presumir que Lark esteja integrado ao código.

6 EXEMPLOS, ANÁLISE E DISCUSSÃO

6.1 Caso sintaticamente e semanticamente válido

Na apresentação, a amostra com cor “incolor”, dureza 7, densidade 2,65 e brilho “vítreo” é apresentada como compatível com quartzo, com quatro de quatro atributos coincidentes. Trata-se de um exemplo ilustrativo de saída esperada, não de uma medição independente de acurácia. O caso evidencia a rastreabilidade entre propriedades declaradas e critérios exibidos no resultado.

6.2 Caso rejeitado

Um segundo exemplo altera a dureza para 11. Embora a atribuição mantenha a estrutura sintática, o valor ultrapassa a faixa definida pela regra semântica. A apresentação mostra a rejeição e a indicação do atributo problemático, exemplificando a utilidade de mensagens de erro específicas para depuração e aprendizagem.

6.3 Execução e testes documentados

O README apresenta duas formas de utilização: execução de um arquivo .stone e passagem direta de propriedades pela linha de comando. A segunda forma exige ao menos uma das propriedades dureza, densidade, cor ou brilho e é descrita como utilizando o mesmo fluxo de análise. Os comandos exemplificados são:

```
python -m drstone.main examples/quartzo.stone
python -m drstone.main --dureza 7 --densidade 2.65 --brilho vitreo --cor incolor
```

O repositório também relaciona arquivos de testes para lexer, parser, semântica, interpretador, minerais e ponto de entrada. A presença desses arquivos na estrutura documentada não permite concluir quantos testes passam, sua cobertura ou o desempenho do sistema sem executar a suíte e registrar resultados reproduzíveis.

6.4 Limitações e ameaças à validade

As restrições deliberadas da DSL — ausência de cálculos, estruturas de controle e funções do usuário — favorecem a aprendizagem, mas reduzem a expressividade. A comparação por atributos físicos depende da qualidade do catálogo e de regras de equivalência e tolerância, que exigem documentação detalhada e avaliação com amostras de referência. Há ainda limitações metodológicas: os casos da apresentação ilustram o comportamento esperado, enquanto este estudo não verifica experimentalmente a implementação, a cobertura de testes ou a acurácia das identificações.

Como evolução, recomenda-se registrar a versão e o commit do código avaliado, executar os testes automatizados, publicar casos de erro por etapa, definir tolerâncias para densidade e dureza quando pertinentes e elaborar um conjunto de amostras rotuladas por fontes mineralógicas confiáveis. Esses procedimentos permitiriam distinguir correção do compilador de qualidade da classificação mineral.

7 CONCLUSÃO

O Dr. Stone apresenta uma proposta didática coerente de linguagem de domínio específico para descrição de minerais. Sua sintaxe declarativa e sua arquitetura em etapas oferecem um contexto concreto para compreender tokens, gramática, AST, validação semântica e interpretação. A separação entre os módulos permite discutir responsabilidades técnicas e demonstrar como restrições de domínio são verificadas além da sintaxe.

A análise documental evidencia exemplos de entradas válidas e inválidas, um fluxo de execução descrito em Python e a intenção de produzir identificação por compatibilidade. Não há, nas fontes examinadas para este artigo, base suficiente para declarar precisão estatística ou desempenho medido. Assim, a principal contribuição demonstrável é educacional e arquitetural. Trabalhos futuros devem consolidar a implementação, executar avaliações reproduzíveis e documentar a qualidade do catálogo e dos critérios de comparação.

REFERÊNCIAS

AHO, Alfred V. et al. Compilers: principles, techniques, and tools. 2. ed. Boston: Pearson Addison-Wesley, 2006.

FOWLER, Martin. Domain-specific languages. Boston: Addison-Wesley, 2010.

GRUNE, Dick; JACOBS, Ceriel J. H. Parsing techniques: a practical guide. 2. ed. New York: Springer, 2008.

PYTHON SOFTWARE FOUNDATION. Python 3 documentation. [S. l.], 2026. Disponível em: https://docs.python.org/3/. Acesso em: 3 nov. 2026.

DRSTONE. Especificação da linguagem DrStone: versão 0.1. GitHub, 2026. Disponível em: https://github.com/Rub3n1t0/DrStone/blob/main/ESPECIFICACAO.md. Acesso em: 3 nov. 2026.

DRSTONE. DrStone: repositório de código e README. GitHub, 2026. Disponível em: https://github.com/Rub3n1t0/DrStone. Acesso em: 3 nov. 2026.

SILVA, Ana Luiza S.; SANTOS, Kauan; MARCOS, Riglei; BOLOGNESI, Rubens; PIRES, Vagner. Dr. Stone: uma linguagem científica para identificar minerais, gemas e joias. Apresentação acadêmica. Disciplina Projeto e Implementação de Compiladores, 2026.